



CCODEL
RAISING A NEW GENERATION OF
GLOBAL LEADERS

Computer Programming I (Java 1)

COS221

Mr. Otavie Okuoyo

Phd (in view Computer Science)

College of Science and Technology



Visit Our Website

ccodel.covenantuniversity.edu.ng



MODULE SIX: OBJECT-ORIENTED PROGRAMMING (OOP)

UNIT ONE: OBJECT-ORIENTED PROGRAMMING (OOP) PART I

OBJECT-ORIENTED PROGRAMMING (OOP) PART I

Learning Objectives

By the end of this unit, you will be able to:

- **define Object-Oriented Programming (OOP) and describe its purpose.**
- **explain the key principles of OOP, including classes, objects, encapsulation, and abstraction.**
- **create basic Java classes and instantiate objects.**
- **implement encapsulation by using appropriate access modifiers.**
- **apply abstraction through method design and class structure.**

Learning Outcomes

At the end of this unit, you should be able to:

- **understand the meaning of Object-Oriented Programming.**
- **define and explain the key principles of OOP: classes, objects, encapsulation and abstraction.**
- **create simple Java classes and objects.**
- **demonstrate encapsulation using access modifiers.**
- **apply abstraction using methods and class design.**

Introduction

- In this unit, we begin exploring Object-Oriented Programming (OOP), a core concept in Java and a key paradigm in modern software development.
- OOP focuses on structuring code using classes and objects, enabling reusable, modular, and scalable applications.
- You will also learn core principles like encapsulation (limiting direct data access) and abstraction (highlighting key features while hiding details). Mastering these basics sets the stage for advanced topics.

What is Object-Oriented Programming?

- **Object-Oriented Programming (OOP) is a paradigm that organizes software design around objects (instances of classes) rather than functions and logic to model real-world entities.**
- **Objects represent real-world entities with attributes (data) and behaviours (methods).**
- **Class serves as template for creating an object.**

Key Concepts of OOP

The key concepts of OOP are (the last four are considered the pillars of OOP):

- i. Class**
- ii. Object**
- iii. Encapsulation**
- iv. Abstraction**
- v. Inheritance (covered in later lessons)**
- vi. Polymorphism (covered in later lessons)**

Why is OOP Important?

Importance of OOP include:

- i. Modularity and Reusability:** Organizes code into reusable, self-contained components (such as classes or objects), making it easier to maintain and scale.
- ii. Real-World Modeling:** Represents real-world entities through objects, making it easier to conceptualize.
- iii. Powerful Features:** Supports key principles like inheritance, polymorphism, encapsulation, and abstraction, enabling robust and flexible code design.

Class

- A class is a blueprint or template for creating objects. It defines the structure (which includes the attributes (variables) and behaviour (methods)) of an object. The syntax is:

```
accessModifier class ClassName {  
    // Fields (attributes)  
    // Methods (behaviour)  
}
```

Class - Example 1 (Person Class)

The code below represent a screenshot of a *Person* class that contains two fields *name* (String) and *age* (int) with a method *introduce()* that prints the person's details.

```
public class Person {  
    // Attributes (fields)  
    String name;  
    int age;  
  
    // Method (Behaviour)  
    void introduce() {  
        System.out.println("My name is " + name);  
        System.out.println("I am " + age + " years old");  
    }  
}
```

Class - Example 2 (Car Class)

- ***Car*** is a class representing a concept of a car. The attributes (or fields) ***model***, ***color***, and ***year*** define the properties that each Car object will have.
- The method ***start()*** defines a behavior that a Car can perform, in this case, printing "***Car started!***".

```
public class Car {  
    String model, color;  
    int year;  
  
    void start() {  
        System.out.println("Car started!");  
    }  
}
```

What is OOP?

- An object is a specific instance of a class. Once you have defined a class, you can create objects from it. The syntax is for creating an object is:

ClassName ***objectName*** = ***new*** ***ClassName()***;

What is OOP? - Ex. 1 (Student Object)

```
public class Student {  
    String name = "Alice"; // Default name  
    int age = 20;           // Default age  
  
    public static void main(String[] args) {  
        Student s1 = new Student(); // Creates class object  
        s1.name = "Nifemi"; // Re-assignment of value to name  
        System.out.println("Student's name: " + s1.name);  
        System.out.println("Student's age: " + s1.age);  
    }  
}
```

The code above represents a ***Student*** class that contains two fields: ***name*** (String) and ***age*** (int). A Student object is created, and the ***name*** property is reassigned a new value, and both the ***name*** and ***age*** values are printed to the console.

What is OOP? - Ex. 2 (Car Object)

```
public class Car {  
    String make = "Toyota";  
    String model = "Corolla";  
  
    public static void main(String[] args) {  
        Car myCar = new Car(); // Create Car object  
  
        System.out.println("Make: " + myCar.make);  
        System.out.println("Model: " + myCar.model);  
    }  
}
```

In the code above, a **Car** class with make and model fields is created. In the **main()** method, an object **myCar** is created, and its properties are accessed and printed. The output is:

Make: Toyota
Model: Corolla

What is OOP? - Ex. 3

```
public class Student {  
    String name;  
    int age;  
  
    void displayInfo() {  
        System.out.println("Hi there! I am " + name);  
        System.out.println("I am " + age + " years old.");  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        s1.name = "Lara";  
        s1.age = 20;  
        // Call the displayInfo method  
        s1.displayInfo();  
    }  
}
```

What is OOP? - Ex. 3

- The java program above defines a *Student* class with two fields: *name* and *age*, and a method *displayInfo()* that prints the details of the student.
- In the *main()* method, an object *s1* of Student is created, its *name* is set to "*Lara*" and *age* to *20*, and the *displayInfo()* method is called to display the information.

Constructor

A constructor is a special method used to initialize objects of a class. It is called automatically when an object is created.

The following are key features of a constructor:

- i. The name of a constructor must be the same as the class.**
- ii. It does not have a return type not even void.**
- iii. If none is written, Java provides a default constructor.**
- iv. You can have multiple constructors in one class (constructor overloading).**

Default Constructor - Example 1

A default constructor is a constructor that takes no parameters. If you do not define any constructor in your class, Java automatically provides a default constructor.

```
class Book {  
    String title;  
    int page;  
    // No Constructor defined  
}  
  
public class DefaultConstructor {  
    public static void main(String[] args) {  
        Book b = new Book();  
        System.out.println(b.title);  
        System.out.println(b.page);  
    }  
}
```

Default Constructor - Example 1 (2)

- In the code above, the **Book** class does not define a constructor, so Java automatically provides a default constructor.
- When you create a Book object with new **Book()**, the default constructor is used. When the code is ran, **title** defaults to **null** and **page** defaults to **0**. The output is:

null

0

Default Constructor - Example 2

```
class Car {  
    String brand, color;  
    Car() {  
        brand = "Ford";  
        color = "red";  
    }  
}  
  
public class DefaultConstructorTwo {  
    public static void main(String[] args) {  
        Car c = new Car();  
        System.out.println("Brand: " + c.brand);  
        System.out.println("Colour: " + c.color);  
    }  
}
```

Default Constructor - Example 2 (2)

- The code above demonstrates a manually defined default constructor. The *Car* class includes a no-argument constructor that initializes the *brand* and *year* fields with default values.
- When an object is created using `new Car()`, the constructor sets the *brand* to "*Toyota*" and the *year* to *2020*. This allows you to control the initial state of objects.

Parameterized Constructor - Ex 1

A parameterized constructor is a constructor that accepts one or more arguments. It allows you to initialize an object with specific values at the time of creation, rather than relying on default or hardcoded values.

```
class Car {  
    String brand;  
    int year;  
  
    // Parameterized Constructor  
    Car(String b, int y) {  
        brand = b;  
        year = y;  
    }  
  
    void display() {  
        System.out.println("Brand: " + brand);  
        System.out.println("Year: " + year);  
    }  
}
```

Parameterized Constructor - Ex 1 (2)

```
public class ParaConstructor {  
    public static void main(String[] args) {  
        Car c1 = new Car("BMW", 2025);  
        Car c2 = new Car("Mercedes", 2022);  
  
        c1.display();  
        c2.display();  
    }  
}
```

- The **Car** class has two fields: **brand** and **year**. Instead of assigning default values, a constructor with parameters is used to initialize each Car object with custom values.
- In the **main()** method: Two Car objects (**c1** and **c2**) are created with different brand names and years.

Encapsulation

Encapsulation is the mechanism of bundling data (fields) and methods (behavior) into a single unit (class) while restricting direct access to some components.

How to Achieve Encapsulation

To achieve encapsulation:

- i. Declare fields as private (restrict direct access).**
- ii. Provide public getter/setter methods to control access.**

Why Encapsulation

The following are reasons for encapsulation:

- i. Data Hiding: Prevents unauthorized access/modification.**
- ii. Validation: Setters can validate input (e.g., prevent negative balance).**
- iii. Flexibility: Internal logic can change without affecting other code.**

Encapsulation - Ex 1

In the *BankAccount* class in the code below, *accountNumber* and *balance* are private, preventing direct access and protecting the integrity of the data. Public methods like *getBalance()*, *deposit()*, and *withdraw()* control how the balance is accessed and modified, ensuring only valid operations (e.g., positive deposits, withdrawal not exceeding balance).

Encapsulation - Ex 1 (2)

```
public class BankAccount {  
    // Private fields (hidden from outside)  
    private String accountNumber;  
    private double balance;  
  
    // Public constructor  
    public BankAccount(String accountNo, double bal) {  
        this.accountNumber = accountNo;  
        if (balance >= 0) {  
            this.balance = bal;  
        }  
    }  
  
    // Public getter (read-only access)  
    public double getBalance() {  
        return balance;  
    }  
}
```

Encapsulation - Ex 1 (3)

```
// Public setter (controlled modification)
public void deposit(double amount) {
    if (amount > 0) {
        balance += amount;
    }
}

// Public method (secure withdrawal)
public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
        balance -= amount;
    } else {
        System.out.println("Invalid amount!");
    }
}
}
```

Encapsulation - Ex 1 (4)

```
public class BankAccountMain {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount("12345", 1000);  
        acc.deposit(500);  
        acc.withdraw(200);  
        System.out.println("Balance: $" + acc.getBalance());  
        // acc.balance = -1000; Error: balance is private!  
    }  
}
```

Abstraction

Abstraction is the process of hiding internal implementation details and showing only the essential features of an objects.

Basic Abstraction Using Methods

Method-level abstraction hides complex logic inside private methods and exposes only essential functionality through simple, public method calls.

Abstraction

- Ex 1

CarStartSystem class demonstrates abstraction, where the ***start()*** method provides an interface for the user to start the car, while steps like turning on the engine, checking fuel, and starting movement are hidden in private methods.

```
class CarStartSystem {  
    // public method abstracts complex internal steps  
    public void start() {  
        System.out.println("Starting the car...");  
        turnEngineOn();  
        checkFuel();  
        startMoving();  
    }  
  
    // These methods are hidden from the user  
    private void turnEngineOn() {  
        System.out.println("Turning Engine On");  
    }  
  
    private void checkFuel() {  
        System.out.println("Checking Fuel");  
    }  
  
    public void startMoving() {  
        System.out.println("Car is Moving!");  
    }  
}  
  
public class CarStartSystemMain {  
    public static void main(String[] args) {  
        CarStartSystem car = new CarStartSystem();  
        car.start();  
    }  
}
```

Abstraction

- Ex 2

The code shows method-level abstraction in the ATM class, where the ***withdrawCash()*** simplifies the cash withdrawal process. Internal steps like inserting the card, entering the PIN, and dispensing cash are hidden in private methods, allowing users to interact only with the high-level ***withdrawCash()***.

```
class ATM {  
    public void withdrawCash() {  
        insertCard();  
        enterPin();  
        processTransaction();  
        dispenseCash();  
    }  
  
    private void insertCard() {  
        System.out.println("Card inserted.");  
    }  
  
    private void enterPin() {  
        System.out.println("PIN entered.");  
    }  
  
    private void processTransaction() {  
        System.out.println("Processing transaction");  
    }  
  
    private void dispenseCash() {  
        System.out.println("Cash dispensed.");  
    }  
}  
  
public class ATMOperation {  
    public static void main(String[] args) {  
        ATM atm = new ATM();  
        atm.withdrawCash();  
    }  
}
```

Abstraction Using Abstract Class

An abstract class cannot be instantiated directly. It provides partial implementation of methods and leave other methods abstract.

Example 1:

Vehicle, in the code below, is an abstract class that has an abstract method *start()* (no implementation). The *Car* and *Bike* classes must provide their own implementations of *start()*. The *stop()* method is already implemented in the *Vehicle* class.

Abstraction Using Abstract Class (2)

```
abstract class Vehicle {  
    // Abstract method (no body)  
    public abstract void start();  
  
    // Concrete method  
    public void stop() {  
        System.out.println("Vehicle stopped!");  
    }  
}  
  
class Car extends Vehicle {  
    @Override  
    public void start() {  
        System.out.println("Car started!");  
    }  
}
```

Abstraction Using Abstract Class (3)

```
class Bike extends Vehicle {  
    @Override  
    public void start() {  
        System.out.println("Bike started!");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Vehicle myCar = new Car();  
        myCar.start(); // Prints: Car started!  
        myCar.stop(); // Prints: Vehicle stopped!  
  
        Vehicle myBike = new Bike();  
        myBike.start(); // Bike started!  
    }  
}
```

Abstraction Using Interfaces

Abstraction using interfaces allows you to define a set of methods that must be implemented by any class that uses the interface, without specifying how those methods should work. This hides the implementation details and focuses on what an object can do, not how it does it.

Example 1: Startable Interface

The *Startable* interface defines a contract with a single method *start()*. The Car and Bike classes both implement this interface and provide their own version of the *start()* method.

Abstraction Using Interfaces (2)

```
interface Startable {  
    void start(); // Abstract method, no body  
}  
  
class Car implements Startable {  
    @Override  
    public void start() {  
        System.out.println("Starting the car");  
    }  
}  
  
class Bike implements Startable {  
    @Override  
    public void start() {  
        System.out.println("Starting the bike");  
    }  
}
```

Abstraction Using Interfaces (3)

```
public class StartableMain {  
    public static void main(String[] args) {  
        Startable myCar = new Car();  
        myCar.start(); // Prints: Starting the car  
  
        Startable myBike = new Bike();  
        myBike.start(); // Prints: Starting the bike  
    }  
}
```

Abstraction Using: Interface Vs Abstract Class

Feature	Interface	Abstract Class
Purpose	Defines a contract for the implementing class.	Provides both a contract and some common behaviour.
Inheritance	A class can implement multiple interfaces.	A class can extend only one abstract class.
Implementation Requirement	Must implement all methods defined in the interface.	Can inherit some methods (from abstract class).
Access Modifiers for Methods	All methods are implicitly public.	Can have different access modifiers for methods.
Use Case	For defining behaviour for unrelated classes.	For related classes with shared functionality.

In-Text Questions (ITQs)

- 1. Why is encapsulation important in Java programming?**
- 2. How does abstraction simplify code for users?**
- 3. What is the difference between an abstract class and an interface?**
- 4. How can you demonstrate encapsulation in a Java class?**

In-Text Answers (ITAs)

- 1. Encapsulation helps protect data by restricting direct access to class fields using private access modifiers and exposing controlled access through public getter and setter methods. This ensures data integrity and supports validation logic.**
- 2. Abstraction hides internal implementation details and shows only essential features. It allows users to interact with simple interfaces or methods without worrying about the underlying complexity, making code easier to use and maintain.**

In-Text Answers (ITAs)

- 3. An abstract class can contain both fully implemented and abstract methods, and it allows for shared functionality among related classes. An interface, on the other hand, defines a contract of methods that must be implemented and is typically used for unrelated classes.**
- 4. Encapsulation is demonstrated by declaring class fields as private and providing public getter and setter methods to access and modify the data, such as in a BankAccount class where balance is modified only through methods.**

Conclusion

- In this unit, you have been introduced to the core principles of Object-Oriented Programming (OOP). You explored how OOP helps organize code more effectively through classes and objects, and learned how to implement key concepts such as encapsulation and abstraction. Through practical examples, you gained experience creating classes, constructors, and methods, and saw how to control access to data and simplify complex operations.
- These concepts lay the groundwork for more advanced topics, including inheritance and polymorphism, which will be covered in upcoming units.

Summary

- **This unit introduced the foundational concepts of Object-Oriented Programming (OOP) in Java. It explained how OOP helps structure code around classes and objects, enabling modularity and reusability.**
- **Core principles covered include encapsulation, where data is hidden using private fields and accessed via public methods, and abstraction, which exposes essential features while hiding complex implementation.**
- **You also learned how to create objects, use constructors (default and parameterized), and apply abstraction through methods, abstract classes, and interfaces.**